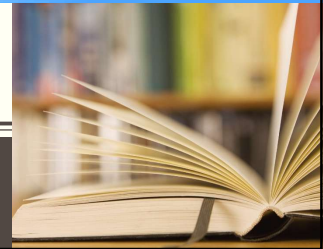
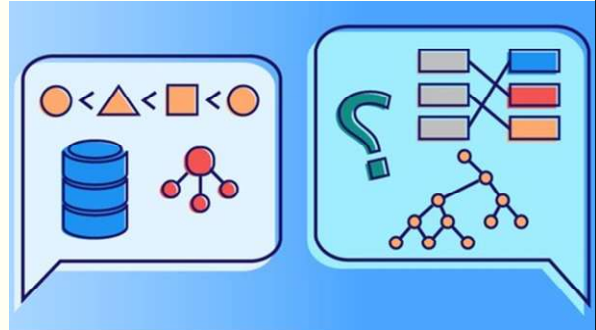


ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 08/06 – Parte 1

- Composición de funciones
- Función Factorial
- Pasaje de parámetros por Referencia
- Uso de “const” en el pasaje de parámetros.
- Funciones de números aleatorios.
- Funciones recursivas
- Ejercicios de repaso de conceptos

2

Algoritmos y Estructuras de Datos – AEDD – Calendario

Instancias de Evaluación	Parte	Fecha
Parcial 2	Parte Teórica Escrita	26/06/2026
	Parte Práctica Escrita	26/06/2026
	Parte Sitio Juez	22/06/2026
Parcial 2 - Recuperatorio	Parte Práctica Escrita	21/08/2026
	Parte Sitio Juez	17/08/2026
Parcial 3	<i>En el mes de septiembre (final) (fecha a confirmar)</i>	

3

Algoritmos y Estructuras de Datos – AEDD – Asistencia 08/06

Contraseña del estudiante
s4msfk

4

Composición de funciones

- Ejemplo: Se dispone de la función `maxim2` que obtiene el máximo entre 2 valores. Emplearla en un programa para calcular el máximo entre 4 valores enteros `a`, `b`, `c` y `d`.

```
int maxim2(int x, int y);

int main() {
    int a, b, c, d, w, z;
    cout << "Ingrese dos enteros: ";
    cin >> a >> b >> c >> d;
    w = maxim2(maxim2(a, b), maxim2(c, d));
    cout << endl << "El maximo es: " << w;
    z = maxim2(a, maxim2(b, maxim2(c, d)));
    cout << endl << "El maximo es: " << z;
    return 0;
}

int maxim2(int x, int y) {
    int max = x;
    if (y > max)
        max = y;
    return max;
}
```

Ingrese dos enteros: 12 7 23 4
El maximo es: 23
El maximo es: 23
<< El programa ha finalizado: codigo de salida:
<< Presione enter para cerrar esta ventana >>

Llamadas anidadas

`maxim2 (maxim2(A, B), maxim2(C,D))` o bien,
`maxim2 (A, maxim2(B, maxim2(C,D)))`

5

Función Factorial

- Realizar una función factorial, que calcule el factorial de un numero entero positivo ingresado.

```
1 #include <iostream>
2 using namespace std;
3
4 float factorial(unsigned int numero);
5
6
7 int main() {
8     unsigned int num;
9
10    cout << "Ingrese un numero natural: ";
11    cin >> num;
12    cout << endl << "El factorial del numero " << num << " es: " << factorial(num) << endl;
13    return 0;
14 }
15
16 float factorial (unsigned int numero) {
17     float resu = numero;
18     if ( numero == 1 )
19         return 1;
20     while (numero > 1) {
21         resu *= --numero;
22     }
23     return resu;
24 }
```

6

Parámetros por referencia

- Son parámetros de entrada/salida
- Se identifican con un ampersand (&) adelante.
- Al momento de la llamada se utilizan para ingresar los valores a trabajar y, cuando acaba el subprograma, son utilizados para devolver los resultados.

7

Uso de “const” en el pasaje de parámetros

- A veces se quiere pasar por referencia un determinado parámetro, pero se pretende que no se pueda cambiar dentro de la función porque no se requiere.
- Para asegurar que dentro de una función no se producen cambios de un parámetro pasado por referencia solamente se tiene que usar la palabra reservada const, para modificar la referencia, dejándola como referencia a constante de la siguiente forma:

```
void referencias(int& a, const int& b)
{
    a = b + 3; // Correcto a es una referencia
    b = b + 1; // Incorrecto 'b' es una referencia a constante
}
```

- En el primer caso la operación de asignación no tiene ningún problema: a puede ser modificada y b puede ser consultada, en cambio en el segundo caso, el compilador generará un error y nos informará de que estamos intentando modificar una constante.

8

Funciones de números aleatorios

Función rand()

- Definida en la biblioteca <stdlib>. Retorna un número "aleatorio" entre 0 y RAND_MAX (como máximo 32767)

Ejemplo: `i = rand();`

- Pseudoaleatorio: Secuencia preestablecida de números "random" pues la semilla del generador de números aleatorios es siempre la misma. Siempre devuelve la misma secuencia para cada llamada a la función.

Llevando a escala

- `rand() % n` retorna un número entre 0 y $n - 1$
- Para obtener un número aleatorio entre 1 y n : `(rand() % n + 1)`
- Sumamos 1 para obtener un número aleatorio entre 1 y n : `(rand() % 6 + 1)` obtiene un número aleatorio entre 1 y 6.

9

Funciones de números aleatorios

Función srand (semilla)

- Definida en la biblioteca <stdlib>
- Inicializa el generador de números aleatorios.
- Se utiliza para fijar el punto de comienzo para la generación de series de números aleatorios; este valor se denomina semilla.

`srand( time( NULL ) ); /* incluir <ctime> */`

`time(NULL)`

- Retorna la hora actual en segundos.
- "Randomiza" la semilla

10

Funciones de números aleatorios: Ejemplo

```
#include <iostream>
#include <cstdlib>

using namespace std;

int main(){
    for (int i=1;i<=20;i++){
        /* números aleatorios entre 1 y 6 */
        cout << (rand() % 6 + 1 ) << " ";

        /* si el contador es divisible por 5, nueva línea */
        if (i % 5 == 0){
            cout << endl;
        }
    }
    return 0;
}
```

```
6 6 5 5 6
5 1 1 5 3
6 6 2 4 2
6 2 3 4 1

<< El programa ha finaliz
<< Presione enter para ce
```

11

Ejercicio

- Escribir un programa que genere aleatoriamente

5 valores enteros e informe:

la cantidad de números impares, y

la cantidad de números de 4 dígitos.

Para ello, deberá implementar 2 funciones:

esImpar(x) que determine si x es impar, y

cantDig(x) que determine la cantidad de dígitos de x (siendo x un número entero).

12

Funciones Recursivas

La recursividad es una técnica muy potente para la resolución de problemas.

- Es adecuada para problemas que se pueden solucionar usando el mismo algoritmo con versiones más reducidas de los datos.
- En general se obtienen soluciones más elegantes y simples que si se lo soluciona de manera iterativa.
- Existen problemas que por su naturaleza tienen un esquema recursivo o recurrente.

13

Recursividad/Recurrencia

Multiplicación de números naturales	$n^*k = n + n^*(k-1)$	con $n^*1=1$
Factorial de un número	$n! = n * (n-1)!$	con $0!=1$
Potencia natural de un número	$a^n = a * a^{(n-1)}$	con $a^0= 1$
Suma de los n primero naturales	$Suma (1,n) = n + Suma (1,n-1)$	con $Suma (1,1) = 1$
Cantidad de dígitos de un número	CantDigitos (n) = 1+ CantDigitos (int (n/10)); con CantDigitos (k) = 1 si $0 \leq k \leq 9$	
Suma de los dígitos de un número	SumaDigitos (n) = (n%10) + SumaDigitos (int (n/10)) con SumaDigitos (k) = k si $0 \leq k \leq 9$	

14

Funciones Recursivas

- Son funciones que se llaman a sí mismas
- Necesitan tener al menos un caso base (se calcula la solución en forma directa, devolviendo un resultado)
- Caso general (no se ha encontrado la solución), luego la función llama a una nueva copia de sí misma (paso de recursión) con un tamaño de problema menor.
- Eventualmente se llega al caso base: esto permite el camino de retorno y la resolución del problema completo.
- El diseño debe garantizar la llegada a uno de los casos base en algún momento.

15

Recursión Vs Iteración

Repetición

- Iteración: utiliza en forma explícita una estructura de repetición
- Recursión: llamadas repetidas a funciones

Terminación

- Iteración: cuando falla la condición del ciclo
- Recursión: cuando se alcanza el caso base
- Ambas pueden tener ciclos infinitos
 - Iteración: Mayor esfuerzo y cantidad de líneas de código que la recursión.
 - Recursión: Costo muy elevado de performance (Las llamadas recursivas requieren tiempo y consumen memoria adicional)

Balance

- Elección entre performance (iteración) y buenas prácticas de ingeniería de software (recursión).

16

Recursión Infinita

Si no se llega a una llamada que no implica recursión (caso base), cada llamada recursiva produce otra llamada recursiva. Una llamada a la función se ejecutará, en teoría, eternamente.

- Esto se llama recursión infinita.
- En la práctica, una función así se ejecutará hasta que la computadora se quede sin recursos, y el programa terminará anormalmente.

17

Repensemos como funciones recursivas

Funcion Factorial.

Funcion cantDigitos.

18

CantidadDigitos - Recursiva

```
int CantidadDigitos(int numero) {  
    if (numero < 10 ) return 1;  
    else return 1 + CantidadDigitos(numero/10);  
}
```

19

SumarDigitos - Recursiva

```
int SumarDigitos(int numero) {  
    if (numero < 10 ) return numero;  
    else return numero%10 + SumarDigitos(numero/10);  
}
```

20

Fibonacci - Recursiva

```
int fibonacci(int numero) {
    if (numero == 0)
        return 0;
    else if (numero == 1)
        return 1;
    else
        return fibonacci(numero - 1) + fibonacci(numero - 2);
}
```

21

EsPotenciaDeDos - Recursiva

Si un número n se puede dividir sucesivamente entre 2 hasta llegar a 1, y nunca da un número impar (excepto al final), entonces es potencia de 2.

```
bool esPotenciaDeDos(int n) {
    if (n == 1) return true;           //  $2^0 = 1$ 
    else if (n < 1 || n % 2 != 0)
        return false; // si es menor que 1 o impar (excepto el 1), no es potencia de 2
    else
        return esPotenciaDeDos(n / 2);
}
```

22

Algoritmos y Estructuras de Datos – AEDD – Asistencia 08/06

Contraseña del estudiante
s4msfk

23

AEDD – Otras tareas !!!!!

LEER:

Libro:

Libro de Savitch (cap. 13)

Libro:

Libro de Deitel & Deitel (cap. 6)

24

Algoritmos y Estructuras de Datos – AEDD – Agenda 08/06 – Parte 1

- ✓ Composición de funciones
- ✓ Función Factorial
- ✓ Pasaje de parámetros por Referencia
- ✓ Uso de “const” en el pasaje de parámetros.
- ✓ Funciones de números aleatorios.
- ✓ Funciones recursivas
- ✓ Ejercicios de repaso de conceptos

¡¡OBJETIVOS DEL DIA ALCANZADOS!!

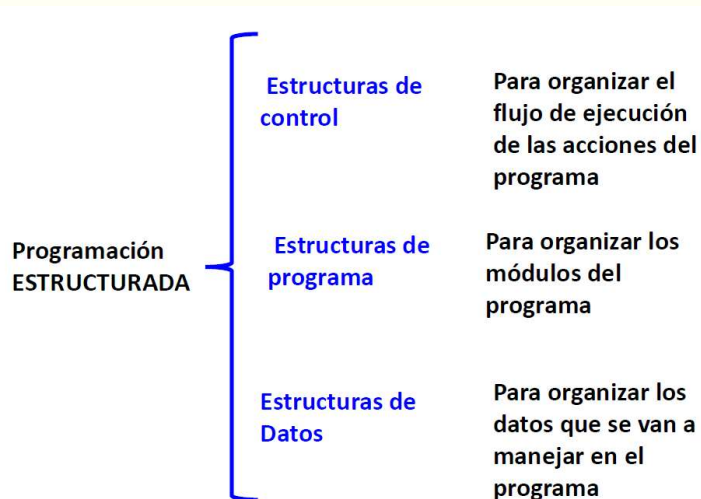
25

Algoritmos y Estructuras de Datos – AEDD – Agenda 08/06 – Parte 2

- Programación Estructurada: Estructuras.
- Estructuras de datos.
- Estructuras de datos: Arreglos.
- Arreglos: Declaraciones.
- Arreglos: Dimensiones.
- Inicializar y referenciar arreglos.
- Primer ejercicio con arreglos.
- Almacenamiento en Memoria.

26

Programación Estructurada



27

Estructuras de datos

- Simples o básicos: caracteres, booleanos, enteros, reales, flotantes.
- Estructurados: colección de valores relacionados. Se caracterizan por el tipo de dato de sus elementos, la forma de almacenamiento y la forma de acceso.

- Estructuras estáticas: Su tamaño en memoria se mantiene inalterable durante la ejecución del programa, y ocupan posiciones fijas.

ARREGLOS - CADENAS - ESTRUCTURAS

- Estructuras dinámicas: Su tamaño varía durante el programa y no ocupan posiciones fijas.

LISTAS - PILAS - COLAS - ARBOLES - GRAFOS.

28

Estructuras de datos: Clasificaciones

- Según donde se almacenan
 - Internas (en memoria principal)
 - Externas (en memoria auxiliar)
- Según tipos de datos de sus componentes
 - Homogéneas (todas del mismo tipo)
 - No homogéneas (pueden ser de distinto tipo)
- Según la implementación
 - Provistas por los lenguajes (básicas)
 - Abstractas (TDA - Tipo de dato abstracto que puede implementarse de diferentes formas)
- Según la forma de almacenamiento
 - Estáticas (ocupan posiciones fijas y su tamaño nunca varía durante todo el módulo)
 - Dinámicas (su tamaño varía durante el módulo y sus posiciones también)

29

Arreglos

- Con frecuencia se puede tener un conjunto de valores, todos del mismo tipo de datos, que forman un grupo lógico o lista de valores.

Por ejemplo, una lista de notas:

NOTAS
30
45
34
68
89

- Una lista simple que contiene elementos individuales del mismo tipo de datos se llama **arreglo unidimensional**.
- Un arreglo unidimensional, el cual también se conoce como arreglo de dimensión única, es una lista de valores relacionados con el mismo tipo de datos que se almacena usando un nombre de grupo único.

30

Estructura de datos: Arreglos

- Secuencia en memoria de un número determinado de **datos del mismo tipo** (**tipo base**: cualquier tipo simple o estructuras definidas por el usuario).
- Los datos se llaman **elementos** (celdas o componentes) del arreglo y se numeran consecutivamente 0,1,2,3,4, etc. Esos números se denominan **índice o subíndice del arreglo** y localizan la posición del elemento dentro del arreglo.
- Los elementos del arreglo se almacenan siempre en posiciones consecutivas de la memoria.
- Los elementos se **acceden en forma directa**, indicando el nombre del **arreglo y el subíndice** (variables indizadas).
- Pueden ser **unidimensionales** (se acceden con un solo índice) o **multidimensionales** (se acceden con varios índices).

31

Arreglos

Nombre del arreglo: Todos los elementos del arreglo tienen el mismo nombre. En el ejemplo, el nombre del arreglo es notas.

notas[0]	34
notas[1]	53
notas[2]	36
notas[3]	76
notas[4]	65
notas[5]	90
notas[6]	33
notas[7]	21
notas[8]	97
notas[9]	79

Los índices de arreglos en C++ siempre tienen límite inferior 0 (indexación basada en cero), y como límite superior el tamaño del arreglo menos 1. Para referirse a un elemento, hay que especificar:

- Nombre del arreglo
- Número de posición del elemento dentro del arreglo

El primer elemento está en la posición 0: notas[0]
 El segundo elemento está en la posición 1: notas[1]
 El quinto elemento está en la posición 4: notas[4]
 El elemento n está en la posición n-1: notas[n-1]

32

Arreglos: Declaración

Los arreglos se deben declarar antes de utilizarse.

Formato de declaración:

tipo-de-dato nombre-del-arreglo [tamaño];

Por ejemplo, para crear un array de nombre *notas*, de diez variables enteras, se lo declara de la siguiente manera:

int notas [10];

El array notas contiene 10 elementos: el primero es notas[0] y el último es notas[9].

33

Arreglos: Declaración

El tamaño de un arreglo puede ser un valor constante o estar representado por una constante:

```
#define TAM 12    // Define una constante para el tamaño del arreglo
int c [TAM];      // Declara el arreglo
```

- **Ejemplos de declaraciones de arrays:**

```
float ingresosMensuales [12]; // Declara un array de 12 elementos float
int notasParcial [60], salariosEmple [1200]; // Declara 2 arrays de enteros
char inicialNombre [45]; // Declara un array de caracteres
float alturaRio [365]; // Declara un array de 365 elementos float
char nombre[15]; // Declara un array de caracteres
```

- **Se puede definir un tipo tabla unidimensional:**

```
typedef float tNotas [30]; // Define un tipo arreglo de 30 elementos float
tNotas curso; // Declara una variable de tipo arreglo de 30 elementos float
```

34

Arreglos: Dimensiones

Un arreglo tiene tres partes:

- La dirección (ubicación en memoria de la primera variable indizada)
- El tipo base del arreglo (determina cuánta memoria –en bytes– ocupa cada variable indizada)
- El tamaño del arreglo (cantidad de elementos del arreglo)

Importante: C++ no comprueba que los índices estén dentro del rango definido.

¡Es responsabilidad del programador!

35

Arreglos: Referencia e inicialización

Referenciar a los elementos del arreglo: Se los puede referenciar por medio del subíndice, o utilizando expresiones con resultado entero:

`c[3] c[j] c[j+3] c[m[i]+1]`

Inicialización de arrays: Antes de utilizarse, los arrays deben inicializarse:

En la declaración:

```
int a [3] = {12,34,45}; /* Declara un array de 3 elementos */
int b [ ] = {4,6,7};    /* Declara un array de 3 elementos */
```

Con asignaciones:

```
for (i=0; i<3; i++) /* Es el método más utilizado. Inicializa
a[i]=0;             todos los valores del array a al valor 0 */
```

Desde el teclado:

```
for (i=0; i<3; i++) /* Inicializa los valores del array a utilizando
cin >> a[i];       los valores ingresados por el usuario */
```

36

Arreglos: Referencia e inicialización

Inicialización de variables de tipo array:

```
typedef int tNuevo[3];
tNuevo a = {12, 34, 45};
tNuevo b = {4, 6, 7};
```

37

Inicializar: Cargar un arreglo

Un arreglo, por ej: int A[6]; :

- Puede cargarse en forma completa al declarar la variable (inicializar):

```
int A[6] = {1,3,5,7,9,11};
```

- Puede cargarse elemento a elemento:

```
A[0]=1; A[1]=3; A[2]=5; A[3]=7; A[4]=9; A[5]=11;
```

Estas sentencias pueden ser consecutivas o estar entre otras sentencias. Incluso pueden estar en cualquier orden.

- Pueden cargarse menos elementos:

```
A[0]=1; A[1]=3; A[2]=5;
```

El resto de los elementos del arreglo serán inicializados en cero. Sólo es válido si los valores a cargar (diferentes de 0) están en posiciones consecutivas.

Si hay una recurrencia, puede cargarse con un ciclo:

```
for (j=0; j<6; j++)
    A[j] = 2*j+1;
```

38

¡¡Pensemos un primer ejercicio!!!

Declarar un arreglo de double de tamaño 7, llamado temperaturas.

Inicializar ese arreglo leyendo 7 valores desde el teclado, cada uno representando la temperatura de un día (pueden ser decimales).

Calcular e imprimir:

- La temperatura promedio de la semana.
- La temperatura mínima y el día en que ocurrió (índice 0 = lunes, ..., 6 = domingo).
- La temperatura máxima y el día correspondiente.
- Contar e informar cuántos días la temperatura superó un umbral definido por el usuario (por ejemplo, 30.0 °C).

39

Arreglos: Almacenamiento en Memoria

Declaración:

`int c [12];` hace que el compilador reserve espacio suficiente para contener 12 valores enteros.

- Los elementos de los arrays se almacenan en bloques contiguos.
- Espacio ocupado por c: 48 bytes, pues posee 12 elementos de tamaño $t = 4$ bytes.

Direcciones:

$$\&c[0] = 1000 = D$$

- $\&c[i] = D + i * t \Rightarrow \&c[4] = 1000 + 4 * 4 = 1016$
- $\&c[i] = D + i * t \Rightarrow \&c[10] = 1000 + 10 * 4 = 1040$

40

Almacenamiento

Muestren las direcciones del arreglo realizado en el ejercicio anterior

41

Algoritmos y Estructuras de Datos – AEDD – Agenda 08/06 – Parte 2

- ✓ Programación Estructurada: Estructuras.
- ✓ Estructuras de datos.
- ✓ Estructuras de datos: Arreglos.
- ✓ Arreglos: Declaraciones.
- ✓ Arreglos: Dimensiones.
- ✓ Inicializar y referenciar arreglos.
- ✓ Primer ejercicio con arreglos.
- ✓ Almacenamiento en Memoria.

¡¡Objetivo cumplido!!!

42

AEDD – Otras tareas !!!!!

Para profundizar en el tema:

Libro:

Resolución de problemas con C++ - Savitch

Capítulo 10

Libro:

C++ para ingeniería y ciencias - Gary J. Bronson

Capítulo 11

43

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¿DUDAS?



44

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¡Buena semana!

